

Chap 1: The MongoDB Data Model

"MongoDB is designed to work with documents without any need of predefined columns or data-types (Unlike Relational databases), making the data model extremely flexible."

The Data Model:

Database → collections → Documents

IMP # Capped Collection:

MongoDB has a concept of capping the collections. This means it stores the documents in the collection in the inserted order. As the collection reaches its limit, the documents will be removed from the collection in FIFO (First In, First Out) order. This means that the least recently inserted documents will be removed first.

The Identifier (-id):

A key, which is nothing but a label, can be roughly compared to the column name in RDBMS. A key is used for querying data from the documents. Hence, like a RDBMS primary key (used to uniquely identify each row), you need to have a key that uniquely identifies each document within a collection. This is referred to as -id in MongoDB.

Polymorphic Schemas:

- Date _____
Page _____
- A polymorphic schema is a Schema where a collection has documents of different types or Schemas.

(a) Object-Oriented Programming:

- for example,

```
// "Document collections" - "HTMLPage" document
{ _id: 1, title: "Hello", type: "HTMLPage",
  text: "<html>Hi.. Welcome to my world </html>" }
.... // Document collection also has a "Picture"
      document.
{ _id: 3, title: "Family Photo", type: "JPEG",
  sizeInMB: 10, .....
}
```

(b) Schema Evolution:

- When you are working with databases, One of the most important considerations that you need to account for is the Schema evolution.
- for example,

```
// "Ticket1" document (Stored in "Tickets" collection)
{
  _id: 1, Priority: "High", type: "Incident",
  text: "Printer not working"
} .....
```

JSON and BSON: MongoDB is a document-based database. It uses Binary JSON for storing its data. MongoDB stores the JSON document in binary-encoded format. This is termed as BSON. The BSON data model is an extended form of the JSON data model.

Chapter 2: Using MongoDB Shell

"MongoDB shell comes with the standard distribution of MongoDB. It offers a JavaScript environment with complete access to the language and the standard functions. It provides a full interface for the MongoDB databases."

Basic querying:

- Query to create or to use in a database:

Query: use <<database-name>>

- use is a keyword and database-name is user-defined name for a database.

Create and insert:

- Explicitly Creating collections:

- There are two ways to create a collection:-

a) Creating a collection with the function.

- CreateCollection(" ")

- db.createCollection("Users")

Here db is the object of the current database and Users-defined is the collection name. This is explicit way of creating collection.

- Implicitly Creating collections:

- This is the process where, we can create collection along with document creation.

Creating a document: Using Prototyping

- Use <<database-Name>>

Example, > Use mydatabase

Switched to db mydatabase

> db

mydatabase

Syntax: db.Users.insert({FName: "Test", LName: "User",
Age: 30, Gender: "M", Country: "US"})

OR

User1 = {FName: "Test", LName: "User", Age: 30,
Gender: "M", Country: "US"}

db.Users.insert(User1)

- To get the list of collection/collections that exist in the database we have,
 - Show collections.
- To retrieve the document/documents from the collection we have, db.<<collection-name>>.find()
Ex. db.users.find() or db.users.find().pretty()
- To display the list of databases we have,
 - Show dbs

Update Method:

- This method is used when the users to change/update the existing value from the document.

- The \$set Operator will be Used for Updating the records.

```
> db.users.update({"Gender": "F"},
                  {"$set": {"Country": "UK"}})
```

- The first parameter is Used to identify the document/ documents.
- The Second parameter is Used to Update the Value Using \$set keyword.
- The third parameter is Used to Update the multiple documents.

Example,

```
db.users.update({"Name": "Test Users"},
                {"$set": {"Country": "UK"}})
```

Q. Write a Query to Update Gender Value to Male where Country Value is US.

```
→ db.users.update({"Country": "US"},
                  {"$set": {"Gender": "Male"}, {multi: true}})
```

- To add a new field in the all existing documents we can use Updating method,

Syntax:

```
db.users.update({}, {"$set": {"Company": "TestComp"}},
                {multi: true})
```

- Update the document with the new field contact with the Value 012345 where the name is "TestUsers".

Syntax:

```
db.users.update({"name": "TestUser"}, {"$set":
    {"contact": 012345}}, {multi: true})
```

Remove Method:

- To remove the bracket delete.
- The field from the existing list of documents, we can use update method with \$unset keyword. consider the following Query where we have deleting the field called company from all the documents.

```
> db.Users.update({}, {$unset: {"company": ""}},  
    {Multi: true})
```

Delete :

- The delete documents in a collection, use the remove() method.
- If you specify a selection Criterion, Only the documents meeting the Criteria will be deleted. If no Criteria is specified, all of the documents will be deleted.
- The following command will delete the documents where Gender = "M":

Selector

```
> db.Users.remove({"Gender": "M"})
```

- The following command will delete all documents.

```
> db.Users.remove({})
```

- If you want to drop the collection, the following command will drop the collections:

```
> db.Users.drop()
```

true

Read:

The `find()` command is used to retrieve data from the database. firing a `find()` command returns all the documents within the collection.

```
> db.users.find()
```

IMP # Query Documents:

- A rich query system is provided by MongoDB. Query documents can be passed as a provided parameter to the `find()` method to filter documents with a collection.
- A query document is specified within open "{" and closed "}" curly braces. A query document is matched against all of the documents in the collection before returning the result set.
- Using the `find()` command without any query document or an empty query document such as `find({})` returns all the documents within the collection.
- A query document can contain selectors and projectors.
- A selector is like a where condition in SQL or a filter that is used to filter out the results.
- A projector is like a the select condition or the selection list that is used to display the data fields.

(1) Selector:

- Date _____
Page _____
- The following command will return the female Users.

```
> db.users.find( {"Gender": "F"},  
{"_id": ObjectId("52f4a826958073ea07e15071"),  
  "Name": "Test User", "Age": 45, "Gender": "F",  
  "Country": "US"}  
-----
```

```
{"_id": ObjectId("52f4a83f958073ea07e15084"),  
  "Name": "Test User 19", "Age": 29, "Gender": "F"}  
-----
```

- Only the documents where the gender field's value is F. We haven't used any projector, this command will return the female users with all the fields.

```
> db.users.find( {"Gender": "F", $or: [{"Country":  
  "India"}]})
```

- The above method will return the female users from India.

```
> db.users.find( {"Gender": "F", $or: [{"Country":  
  "India"}, {"Country": "US"}]})
```

- In this query, we can retrieve the document where the gender value is F, and the country value is either India or US.
- To check the count of the documents of female users which belong to either India or US.

```
> db.users.find().count()
```

Example,

```
> db.users.find( {"Gender": "F", $or: [{"Country":  
  "India"}, {"Country": "US"}] }).count()
```

(2) Projector:

- If we want to see a specific document from a collection. We use Selector, but if we want to see a specific fields and document, along with a specific fields we can use Projector.
- Suppose you want to display the first name and age of all female employees.

Selector

```
> db.users.find( {"Gender": "F"}, {"Name": 1, "Age": 1} )
```

Projector

(a) Sort():

- In MongoDB, the sort order is specified as follows: 1 for ascending and -1 for descending sort.
- The following command:

```
> db.users.find( {"Gender": "F"}, {"Name": 1, "Age": 1} ).sort( {"Age": 1} )
```

(b) limit():

- In huge collections with thousands of documents, if you want to return only five matching documents, the limit command is used, which enables you to do exactly that. The following commands:

```
> db.users.find( {"Gender": "F", $or: [{"country": "India"}, {"country": "US"}] } ).limit(2)
```

(c) Skip():

- Date _____
Page _____
- If the requirement is to skip the first two records and return the third and fourth user, the skip command is used. The following commands need to be executed:

```
>db.Users.find({ "Gender": "F", $or: [{ "country": "India"},  
  { "country": "US" }] }).limit(2).skip(2)
```

(d) findOne():

- The `findOne()` method can take the same parameters as `find()`, but rather than returning a cursor, it returns a single document. The following command:

```
>db.users.findOne({ "Gender": "F" }, { "Name": 1,  
  "Age": 1 })
```

Using Cursor:

- (1) When the `find()` method is used, MongoDB returns the results of the query as a cursor object. In order to display the result, the MongoDB shell iterates over the returned cursor.
- (2) MongoDB enables the users to work with the cursor object of the `find` method. In the next example you will see how to store the cursor object in a variable and manipulate it using a while loop.

(a) explain():

- The `explain()` function can be used to see what steps the MongoDB database is running while executing a query.
- The following are the Verbosity modes: `allPlansExecution`, `executionStats`, and `queryPlanner`. The default Verbosity

mode is queryPlanner, which means if nothing is specified it defaults to queryPlanner.

- The following code covers the steps executed when filtering on the username field:

```
> db.users.find({ "Name": "Test User" }).explain  
("allPlansExecution")
```

Single key Index:

- It is use to create an index on the Name field of the document.

```
> db.testInx.createIndex({ "Name": 1 })
```

```
> db.testInx.find({ "Name": "User101" }).explain  
("allPathsExecution")
```

- This method will be create index of Name field. At the time of something searching instead of scanning entire documents. (including all the fields of document).
- The search operation will only search a Name field in the indexes, wherever, the selector value is match with index field as a result the corresponding document will get return.
- So, it will not scan all the documents.

Compound Index:

- When creating an index, you should keep in mind that the index covers most of your queries. If you

Date _____
Page _____

Sometimes, query Only the Name field and at times you query both the Name and the Age field, creating a compound index on the Name and Age fields will be more benefits than an index that is created on either of the fields because the compound index will cover both queries.

- The following command creates a compound index on fields Name and Age of the collection testindx:

```
> db.testindx.createIndex({"Name": 1, "Age": 1})
```

- compound indexes help MongoDB executes queries with multiple clauses more efficiently.

Support for Sort Operations:

- A compound index needs to be created when sorting on multiple fields. In a compound index, the Output can be in the Sorted order of either an index prefix or the full index.
- The Index Prefix is a subset of the compound index, which contains one or more fields from the start of the index.
- The following commands:

```
> db.testindx.createIndex({"Age": 1, "Name": 1, "Class": 1})
```

```
> db.testindx.find().sort({"Age": 1})
```

```
> db.testindx.find().sort({"Age": 1, "Name": 1})
```

```
> db.testindx.find().sort({"Age": 1, "Name": 1, "Class": 1})
```

Using Conditional Operators:

- Conditional Operators enables you to have more control over the data you are trying to extract from the database.

\$lt and \$lte:

- They stand for "less than" and "less than or equal to" respectively.
- If you want to find all students who are younger than 25 ($\text{Age} < 25$).
> db. Students.find({"Age": {"\$lt": 25}})

\$gt and \$gte:

- The \$gt and \$gte Operators stand for "greater than" and "greater than or equal to", respectively.
- Let's find out all of the students with $\text{Age} > 25$.
> db. Students.find({"Age": {"\$gt": 25}})
- If you change the above example to return students with $\text{Age} \geq 25$, then the command is,
> db. Students.find({"Age": {"\$gte": 25}})

\$in and \$nin:

- Let's find all students who belong to either class C1 or C2. The comp for the same is:
> db. Students.find({"class": {"\$in": ["C1", "C2"]}})

• Cluster - groups of nodes / computers

Q. Say you want to find out all students whose gender is either "M" or they belong to class "c1" and "c2" and whose age is greater than or equal to 25.

```
> db.Students.find({$or:[{"gender": "M", "class": {"$in": ["c1", "c2"]}}, {"Age": {"$gte": 25}}])
```

Regular Expressions: Regular expressions are useful in scenarios where you want to find students with name starting with "A".

- Say you want to find all students with names starting with "St" and "Te" and whose class begins with "che". The same can be filtered using regular expressions, like so:

```
> db.Students.find({"Name": /^(St|Te)*/i, "class": /^(che)/i})
```

MapReduce:

- MapReduce is a framework that is used to process problems that are highly distributed across enormous databases, datasets and are run using multiple nodes.
- If all the nodes have the same hardware, these nodes are collectively referred to as a cluster; otherwise, it's referred to as a grid. This processing can occur on structured data (data stored in a database) and unstructured data (data stored in a file system).

(a) aggregate():

- The aggregation framework enables you find out the aggregate Value with using the MapReduce function.
- To find the count of male and female Students. This can be achieved by executing the following commands:

```
> db.students.aggregate({$group: {-id: "$Gender",  
totalStudent: {$sum: 1}}})
```

```
{ "-id": "F", "totalStudent": 6 }  
{ "-id": "M", "totalStudent": 9 }
```
- To find out the class-wise average score, the following command can be executed:

```
> db.students.aggregate({$group: {-id: "$Class",  
AvgScore: {$avg: "$Score"}}})
```

Sharding:

- One of the following factors when designing the application model is whether to partition the data or not. This is implemented using Sharding in MongoDB.
- Sharding is also referred as partitioning of data. In MongoDB, a collection is partitioned with its documents distributed across cluster of machines, which are referred as Shards.

CHAPTER 3: MongoDB Architecture

"MongoDB architecture covers the deep-dive architectural concepts of MongoDB."

Core Processes: The core components in the MongoDB package are,

- mongod, which is the core database process.
- mongos, which is the controller and query router for sharded clusters.
- mongo, which is the interactive MongoDB shell.

MongoDB Tools:

- Apart from the core services, there are various tools that are available as part of the MongoDB installation:

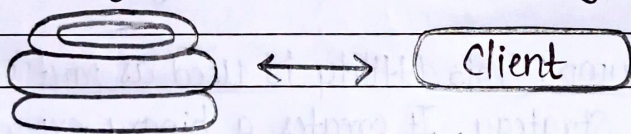
- mongodump: This utility is used as part of an effective backup strategy. It creates a binary export of the database contents.
- mongorestore: The binary database dump created by the mongodump utility is important to a new or an existing database using the mongorestore utility.
- bsondump: This utility converts the BSON files into human-readable formats such as JSON and CSV. For example, this utility can be used to read the output file generated by mongodump.
- mongoimport, mongoexport: mongoimport provides a method for taking data in JSON, CSV, or TSV formats and

importing it into a mongod instance. mongoexport provides a method to export data from a mongod instance into JSON, CSV, or TSV formats.

- mongostat, mongotop, mongosniff: These Utilities provided diagnostic information release related to the current Operation of a mongod instance.

Standalone Deployment:

- Standalone Deployment is used for development purpose; it's doesn't ensure any redundancy of data and it doesn't ensure recovery in case of failure.
- So it's not recommended for use in production environment. Standalone deployment has the following components: a single mongod and a client connecting to the mongod,



~~IMP~~ #

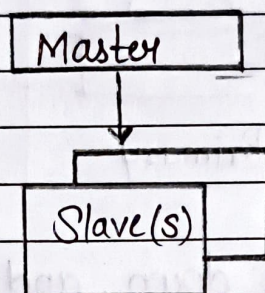
Replication: It provides for data redundancy by a replicating data on different nodes, thereby providing protection of data in case of node failure, Replication provides high availability in a MongoDB deployment.

- There are two types of replication supported in MongoDB: traditional master/slave replication and replica set.

(a) Master/Slave Replication:

- In MongoDB, the traditional Master/Slave replication is available but it is recommended only for more than 50 node replications, the preferred replication approach is

replica Sets. In this type of replication, there is One master and number of Slaves that replicate the data from the master. The only advantages With type of the replication is that there's no restriction on the number of Slaves With-in a cluster. However, thousands of Slaves will burden Over the master node, So in practical Scenarios it's better than to have less than dozen Slaves.

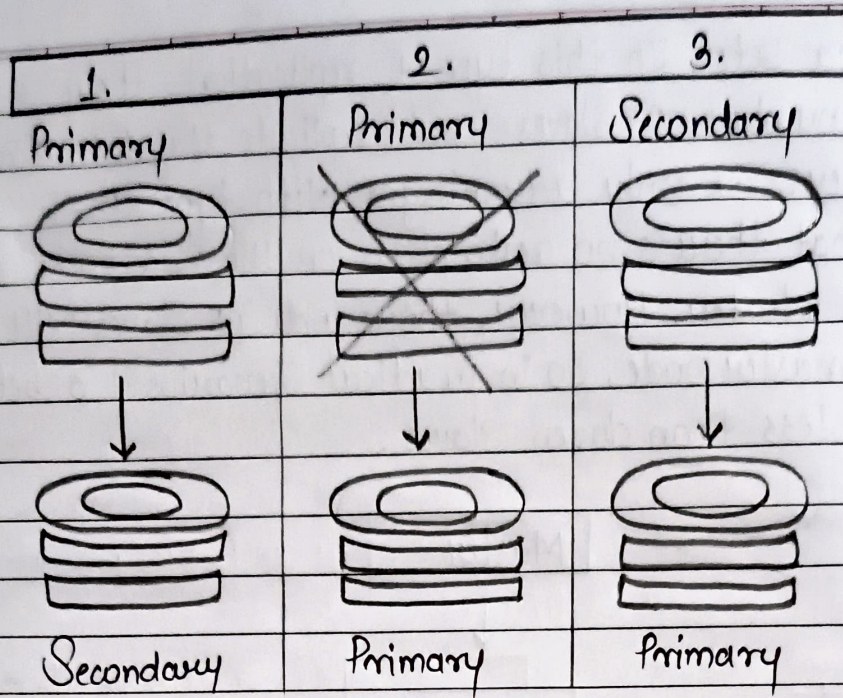


- There are two main reasons behind a Slave becoming Out of Sync:

- (1) The Slave Shuts down or Stops and restarts later. During this time, the Oplog may have deleted the log of Operations required to be applied On the Slave.
- (2) The Slave is slow in executing the Updates that are available from the master.

(b) Replica Set:

- Replica sets are basically a type of master-slave replication but they provide automatic failover. A Replica set has One master, which is termed as Primary, and multiple Slaves, Which are termed as Secondary in the replica set context; however, Unlike master-Slave replication, there's no node of One that is fixed to be Primary in the replica set.



1. The Primary goes down, and the Secondary is promoted as primary.
2. The Original primary comes up, it acts as slave, and becomes the Secondary node.

Primary and Secondary Members:

- (1) Primary Member: A replica set can have only one primary, which is elected by the voting nodes in the replica set. Any node with associated priority as 1 can be elected as a primary.
- (2) Secondary Member: A normal secondary member holds the copy of the data. The secondary member can vote and also can be a candidate for being promoted to primary in case of failover of the current primary.

Types of Secondary Members:

- (1) Priority 0 members are Secondary members that maintain the primary's data copy but can never become a primary in case of a failover. Apart from that, they function as a normal secondary node, and they can participate in voting and can accept read requests.